

Justificación para Importador:

Uso del patrón Strategy

Se aplicó el patrón Strategy para permitir que la lógica de importación de hechos pueda variar dinámicamente según el origen de los datos, sin necesidad de modificar la clase Administrador. Este patrón nos permite definir una interfaz común (ImportadorDeHechos) que representa el comportamiento general de importación, y proveer distintas implementaciones concretas (como ImportadorCSV, ImportadorDesdeAPI, etc.).

Además, el uso del patrón Strategy mejora el desacoplamiento entre clases, separando la responsabilidad de "gestionar" del Administrador y la de "importar" del Importador, respetando así también el principio de responsabilidad única (SRP).

Uso de inyección de dependencias (Dependency Injection)

Para evitar un acoplamiento fuerte entre el Administrador y una implementación concreta de ImportadorDeHechos, se optó por aplicar el principio de inyección de dependencias. En lugar de que Administrador cree internamente el importador que necesita, se le inyecta desde afuera una instancia de la estrategia deseada.

Esto ofrece múltiples ventajas:

- Facilita el testeo, ya que se pueden inyectar versiones simuladas (Fake, Mock) del importador.
- Permite flexibilidad y reutilización, ya que distintos Administradores pueden trabajar con distintos tipos de importadores según el contexto.
- Favorece la extensión, dado que nuevas estrategias de importación se pueden incorporar sin afectar la lógica existente.

Uso del patrón Adapter

El componente importador actúa como un Adapter, ya que encapsula la interacción con una librería externa para el procesamiento de archivos CSV, y adapta esa lógica a nuestro modelo de dominio (Colección, Hecho).

Esto permite desacoplar el sistema del formato de entrada y de la librería específica, facilitando el mantenimiento y eventual sustitución del mecanismo de importación sin afectar el resto del sistema.

Siguiendo el patrón Adapter, se traduce la interfaz del proveedor externo a una interfaz interna amigable para el sistema.

Justificación para enums

Optamos por la utilización de enumeraciones en lugar de clases para representar ciertos conceptos que poseen un conjunto finito, bien definido y acotado en posibles valores. Este sería el caso de *TipoDeVisitante*, *OrigenDeHecho* y *EstadoDeSolicitudDeEliminacion*.

Estas entidades no requieren de comportamientos complejos, lo que hace que el modelado como enum sea más adecuado. Por ejemplo, en el caso de *TipoDeVisitante*, sólo existen dos tipos posibles (VISUALIZADOR y CONTRIBUYENTE). Algo similar ocurre con *OrigenDeHecho*, que puede ser únicamente DATASET, CARGA_MANUAL o CONTRIBUYENTE, y con *EstadoDeSolicitudDeEliminacion*, cuyos estados posibles son PENDIENTE, APROBADA y RECHAZADA.

Por lo tanto, modelar estos elementos con clases, no tendría sentido por una cuestión que se estarían expresando simples valores constantes.

Justificación para map en hechos de colección:

Para los hechos dentro de las colecciones optamos por un map: “...un hecho está repetido si el ‘título’ es el mismo. De ser así, se pisarán los atributos del existente.” Con el map podemos usar el título para comparar y sobrescribir los datos del hecho.